=== and !== are more strict versions of the two-character counterparts (1 !== 1.0 but 1 == 1.0). The three-character comparison operators are not commonly seen in Elixir codebases.

Comparison can be done on any data type ("hi" > 3). This might cause some head-scratching at first, but it simplifies the implementation of data comparison and makes it easier to write sorting functions.

Arithmetic

+, -, *, / are the built-in arithmetic operators. Other math operators (modulo, power, rounding) are implemented in the Integer and Float modules.

Boolean Operators

There are two flavors of boolean operators, just as in Ruby. and, or, and not are strict boolean operators. &&, ||, and ! are truthy boolean operators.

Strict boolean operators expect the first argument to be a boolean. Truthy boolean operators work on any data type. So, 1 && true is valid but 1 and true throws an error. Truthy boolean operators are more commonly used than strict boolean operators.

In Ruby, and has a different precedence than &&. This leads to subtle bugs in certain situations. This isn't the case in Elixir because and and && have the same precedence.

Throw and Raise

Elixir supports throw and raise operators to bubble exceptions through the stack. throw can be used to propagate any Elixir term, but raise is only for Elixir exceptions. In practice, these aren't used often. Most Elixirists prefer to use pattern matching to handle errors in their application. (You'll learn about this in Chapter 4, Pattern Matching Your Way to Success, on page 49.)

Function Piping

The pipe operator |> is heavily used by Elixir programmers. This operator takes a value and passes it as the first argument to a provided function. You use this to create a chain of function calls that's easily readable and well-formatted. Each return value is then piped into the next function. This is similar to method chaining in Ruby.

```
"a string"
|> String.upcase()
|> String.replace("A", "THE")

# "THE STRING"
```